

Day 4 — Polynomial Regression and Regularization

Objective

Understand model capacity and how regularization controls complexity.

Topics and detailed notes

- Polynomial features create nonlinear relationships while the estimator can remain linear in its parameters.
- Increasing polynomial degree increases capacity and can produce severe overfitting.
- Ridge penalizes squared coefficients; Lasso penalizes absolute coefficients.
- Regularization trades a little training fit for better generalization.
- Regularization strength is a hyperparameter and must be selected using validation.
- Feature scaling is particularly important when regularization penalties act directly on coefficient magnitudes.

Mathematics / formulas to know

1. Euclidean distance

For two points:

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

For two dimensions:

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

2. Manhattan distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. Minkowski distance

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

Special cases:

$$p = 1$$

→ Manhattan distance

$$p = 2$$

→ Euclidean distance

4. KNN classification

The prediction is generally the majority class among the k nearest observations:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_k)$$

5. KNN regression

The basic prediction is:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k y_i$$

6. Distance-weighted KNN

A common weighting idea is:

$$w_i = \frac{1}{d_i}$$

Then:

$$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}$$

The closer observations receive more influence.

7. Why scaling matters

Suppose:

$$Age = 20$$

and:

$$Income = 100000$$

Using Euclidean distance directly:

$$d = \sqrt{(Age_1 - Age_2)^2 + (Income_1 - Income_2)^2}$$

The income difference can completely dominate the distance.

This is why feature scaling is especially important for KNN.

Implementation pattern

Baseline linear model → generate polynomial features → validate complexity → add regularization → compare validation performance → final test.

Experiments to perform

- Increase polynomial degree one step at a time.
- Track training vs validation error.
- Vary regularization strength.
- Compare coefficient magnitudes.

End-of-day deliverables

- Demand prediction project.
- Complexity curve.
- Regularization experiment table.
- Explanation of bias/variance tradeoff.

Mastery questions

1. Why can a more complex model have lower training error but worse validation error?
2. Why does Lasso sometimes set coefficients to zero?
3. Why does regularization require careful scaling?

PLAN

Welcome to Day 4, bro! You have been crushing these concepts, and today we are leveling up your architectural skills again.

Before we map out the phases, I need to point out another massive plot twist in your curriculum, just like we saw on Day 3.

The text of your syllabus for Day 4 is entirely about **"Polynomial Regression and Regularization"**. It covers how to make models more complex and how to stop them from memorizing data. However, the actual mathematical formulas provided in your document have absolutely nothing to do with regression lines—they are the exact formulas for **K-Nearest Neighbors (KNN) and Distance Metrics!**

As a Senior Engineer, we adapt to the data we are given. We are going to master *both* the text and the math today. Here is your master plan for Day 4, from theory to practical **sklearn** execution.

Phase 1: The Theory of Model Complexity

Right now, you know how to fit a straight line. But what if you are tracking telemetry for Retro Reload, and you notice that as the number of concurrent players increases, the server load doesn't just go up in a straight line—it curves upwards exponentially?

- **Polynomial Features:** We can create nonlinear relationships while the estimator can remain linear in its parameters. This means we don't have to throw away our Linear Regression engine; we just engineer new features (like X^2 or X^3) and feed them in.
- **The Overfitting Trap:** Increasing the polynomial degree increases capacity and can produce severe overfitting. If you use a polynomial degree of 15, the line will violently zig-zag to touch every single dot perfectly. Your training error drops to zero, but when deployed to production, it fails completely.

Phase 2: The Fix (Regularization)

To fix a zig-zagging, overfitted model, we use Regularization. Regularization trades a little training fit for better generalization. It mathematically forces the engine to keep the weights (w) as small as possible.

- **Ridge Regression (L2):** Ridge penalizes squared coefficients. It shrinks large weights down so no single feature dominates the model.
- **Lasso Regression (L1):** Lasso penalizes absolute coefficients. It is so aggressive that it sometimes sets coefficients to exactly zero, effectively deleting useless features from the model completely!
- **The Scaling Rule:** Feature scaling is particularly important when regularization penalties act directly on coefficient magnitudes. If you don't scale your data, a feature measured in millions will be unfairly penalized compared to a feature measured in decimals.
- **Tuning:** Regularization strength is a hyperparameter and must be selected using validation. We will use K-Fold cross-validation to find the perfect penalty strength.

Phase 3: The Hidden Math Engine (K-Nearest Neighbors)

Here is where we decode the mathematical formulas hidden in your syllabus. KNN doesn't learn an equation; it simply plots all the data in space and makes predictions based on the closest data points.

1. How it measures "Closeness" (Distance Metrics):

- **Euclidean distance:** The standard straight-line distance. For two points:
 $d(x,y)=\sqrt{\sum_{i=1}^n(x_i-y_i)^2}$. For two dimensions:
 $d=\sqrt{(x_1-y_1)^2+(x_2-y_2)^2}$.
- **Manhattan distance:** The grid-like "city block" distance:
 $d(x,y)=\sum_{i=1}^n|x_i-y_i|$.

- **Minkowski distance:** The generalized master formula:

$$d(x,y)=\left(\sum_{i=1}^n\left|x_{\{i\}}-y_{\{i\}}\right|^p\right)^{1/p}$$
. *Special cases:* When $p=1$, it becomes Manhattan distance, and when $p=2$, it becomes Euclidean distance.

2. How it makes Predictions:

- **KNN classification:** The prediction is generally the majority class among the k nearest observations: $\hat{y}=\text{mode}(y_{\{1\}},y_{\{2\}},\dots,y_{\{k\}})$.
- **KNN regression:** The basic prediction is the average: $\hat{y}=\frac{1}{k}\sum_{i=1}^ky_{\{i\}}$.
- **Distance-weighted KNN:** A common weighting idea is $w_{\{i\}}=\frac{1}{d_{\{i\}}}$. Then,

$$\hat{y}=\frac{\sum w_{\{i\}}y_{\{i\}}}{\sum w_{\{i\}}}$$
. The closer observations receive more influence in the final prediction.

3. The Fatal Scaling Flaw: Suppose Age is 20 and Income is 100000. Using Euclidean distance directly, the income difference can completely dominate the distance. This is exactly why feature scaling is especially important for KNN.

Phase 4: Implementation Pattern & Deliverables

We will translate this into Python using the required workflow sequence: Baseline linear model $\rightarrow$ generate polynomial features $\rightarrow$ validate complexity $\rightarrow$ add regularization $\rightarrow$ compare validation performance $\rightarrow$ final test.

Experiments you will execute today:

- Increase polynomial degree one step at a time and track training vs validation error.
- Vary regularization strength and compare coefficient magnitudes.

Your End-of-Day Deliverables:

- A Demand prediction project.
- A visual Complexity curve.
- A Regularization experiment table.
- A written explanation of the bias/variance tradeoff.

We have a lot of ground to cover between complex polynomials, aggressive regularization, and the physical geometry of KNN.